GreensOnly

Classists Project Report

TABLE OF CONTENTS

REPORT

This document outlines the report for our nursery system, including a brief overview, an in-depth discussion of the design patterns used, our GitHub branching strategy and merge rules, and the testing structure.
Link to Google Doc Report:
https://docs.google.com/document/d/1TVBaclArVAIFwFhHV6DCYrGhV-Evjj8llirglfBl5co/edit?tab=t.nl58vtbpe8vs

MEET THE TEAM

What you were in charge of, contributions, short and sweet

Cailin Smith - Abstract Factory, Strategy, Singleton, Staff Commands, Facade, UML Diagrams, Command Line Interface and System Activity Diagram

Jordan Naidoo - Template Method, Decorator, UML Diagrams, GUI, System Activity Diagram.

Edwin Küsel - Season State, Plant State, Mediator Pattern, UML Diagrams, and Report.

Abhay Rooplall - Customer Commands, Chain of Responsibility, UML Diagrams, Report.

Alex Lange - Iterator, Observer, Graphical User Interface, UML Diagrams, Report.

PROJECT OVERVIEW

The GreensOnly nursery management system is a C++-based application that simulates a plant nursery with seasonal inventory, staff roles, and customer interactions using design patterns for extensibility and maintainability. It leverages Abstract Factory to generate season-specific plant families (Spring, Summer, etc.), Strategy for plant care behaviours (pruning, watering), State for lifecycle progression and seasonal transitions, Chain of Responsibility for staff request

handling (NurseryStaff to SalesStaff to Manager), and Command to encapsulate customer actions (checkout, refund, etc.). The Template Method ensures consistent cost calculation across plant types, while Iterator enables safe traversal of seasonal inventory. A Singleton Nursery coordinates global state, and Observer notifies staff of inventory thresholds. The system is accessible via both a Text-Based User Interface (TUI) and Graphical User Interface (GUI), with the TUI leveraging the Facade pattern to simplify interaction with the underlying subsystems.

Assumptions

- ➔ All plants are grown from a seed. This means plants such as ferns, which reproduce via spores, are not considered.
- ➔ It is assumed that the nursery naturally accommodates for each plant's needs in terms of climate, and so this was not explicitly demonstrated in terms of plant separation.
- ➔ Customers' receipts will be collected for order history and refund support
- ➔ All transactions are done in person and not through any online system. This means no delivery or scheduling pick-ups.
- ➔ No equipment (e.g., spades or tools) will be sold; only plants and their decorations will be available for purchase.

RESEARCH

As research for the system was conducted, many online guides and sources were consulted to determine the necessary pruning and watering needs of each plant as well as the season they prefer. For example, Basil prefers high levels of moisture, are best thinned when pruned and grow best in the Summer. This information is essential for a nursery to know to grow and sell plants. The system reflects this by having different watering and pruning strategies for each plant.

Furthermore, research into how the staff are divided in nurseries was conducted. Managers are usually highest in the hierarchy with sales staff and nursery staff below them. The sales staff are often in charge of sales interactions with the customers while nursery staff are in charge of customer-plant interactions.

Further research was conducted into finding a GUI library to make use of for our system. There are many GUI libraries for C++ that were researched. These include Qt, wxWidgets, Dear ImGui and FTXUI. Qt had very good support and a modern appearance. wxWidgets is similar to using native widgets of the OS. Dear ImGUI is lightweight and does not have external dependencies. FTXUI provided a simple but modern interface which was easy to understand and implement. The decision was between Qt and FTXUI, with the latter being chosen since Qt had a steeper learning curve.

References

- Gardeners' World (2024). *How to water house plants*. Available at: https://www.gardenersworld.com/house-plants/how-to-water-house-plant (Accessed: 02 November 2025).
- Royal Horticultural Society (n.d.) *Watering*. Available at: https://www.rhs.org.uk/garden-jobs/watering (Accessed: 02 November 2025).
- UC Marin Master Gardeners (n.d.) *Pruning Cuts*. Available at: https://ucanr.edu/site/uc-marin-master-gardeners/pruning-cuts (Accessed: 02 November 2025).
- wxWidgets (n.d.) *Overview - wxWidgets*. Available at: https://wxwidgets.org/about/ (Accessed: 02 November 2025).
- The Qt Company (n.d.) *Qt Features, Framework Essentials, Modules, Tools & Add-Ons*. Available at: https://www.qt.io/product/features (Accessed: 02 November 2025).
- Cornut, O. (n.d.) *Dear ImGui: Bloat-free Graphical User interface for C++ with minimal dependencies*. GitHub. Available at: https://github.com/ocornut/imgui#readme (Accessed: 02 November 2025).
- Sonzogni, A. (n.d.) *FTXUI*. GitHub. Available at: https://github.com/ArthurSonzogni/FTXUI#features (Accessed: 02 November 2025)

FUNCTIONAL REQUIREMENTS

Functional requirement		Design Pattern	Reasoning behind the choice of design pattern
FR 1	The system will instantiate plant families according to the season using seasonal factories.	Abstract Factory	Enhances adaptability and ensures only seasonally appropriate plants are created.
FR 2	The system will allow staff to handle customer requests based on their level in the hierarchy.	Chain of Responsibility	Allows for scalability and avoids hardcoded requests by distributing handling of requests among employees dynamically.
FR 3	The system will handle nursery operations like watering, pruning etc through command objects that encapsulates the actions.	Command	Encapsulates user actions for clear separation between invokers and receivers.
FR 4	The system will allow plants to be decorated based on customer preference like gift wrapping, arrangements etc.	Decorator	Allows for plant objects to be changed at runtime based on customer requests.
FR 5	The system will allow for traversal of the plant collections e.g. seasonal plants.	Iterator	Provides a uniform way to access elements in plant collections.
FR 6	The system will enable communication between different areas like SalesArea and NurseryArea through a central mediator.	Mediator	Reduces direct coupling between staff and the area ensuring coordinated communication.
FR 7	The system will notify relevant staff members when plant inventories fall below a defined threshold.	Observer	Allows for automatic stock level awareness without constant manual checking.

FR 8	The system will ensure that only one shared instance of the Nursery exists throughout the system runtime, allowing for centralized management of plant data and system state.	Singleton	Allows for centralized management of the Nursery's plant data and overall system state (like season), preventing data inconsistencies..
FR 9	The system will allow for dynamic plant management through growing states, where each state defines growth stage and health of the plant.	State (Growth)	Allows plants to change their behaviour as they progress through growth stages without complex conditional logic.
FR 10	The system will change its behaviour based on the current season, ensuring that seasonal operations such as planting are appropriate.	State (Seasons)	Allows the nursery to adapt its behaviour when the season changes without changing the core logic.
FR 11	The system will use different pruning strategies based on the plant type and its requirements.	Strategy (Pruning)	Allows for easy addition or modification of plant care without affecting other parts of the system.
FR 12	The system will use different watering strategies based on the plant type and its requirements.	Strategy (Watering)	Allows for easy addition or modification of plant care without affecting other parts of the system.
FR 13	The system will provide a simple, unified interface for clients to access the core functionalities of the complex nursery subsystem.	Façade	Reduces complexity for the client by encapsulating the interactions of various subsystem components behind a single class.
FR 14	The system will make use of a step-wise algorithm to calculate the cost based on both the season and plant cost.	Template Method	Allows for the deferring of some steps in the calculation of the cost to the subclasses.

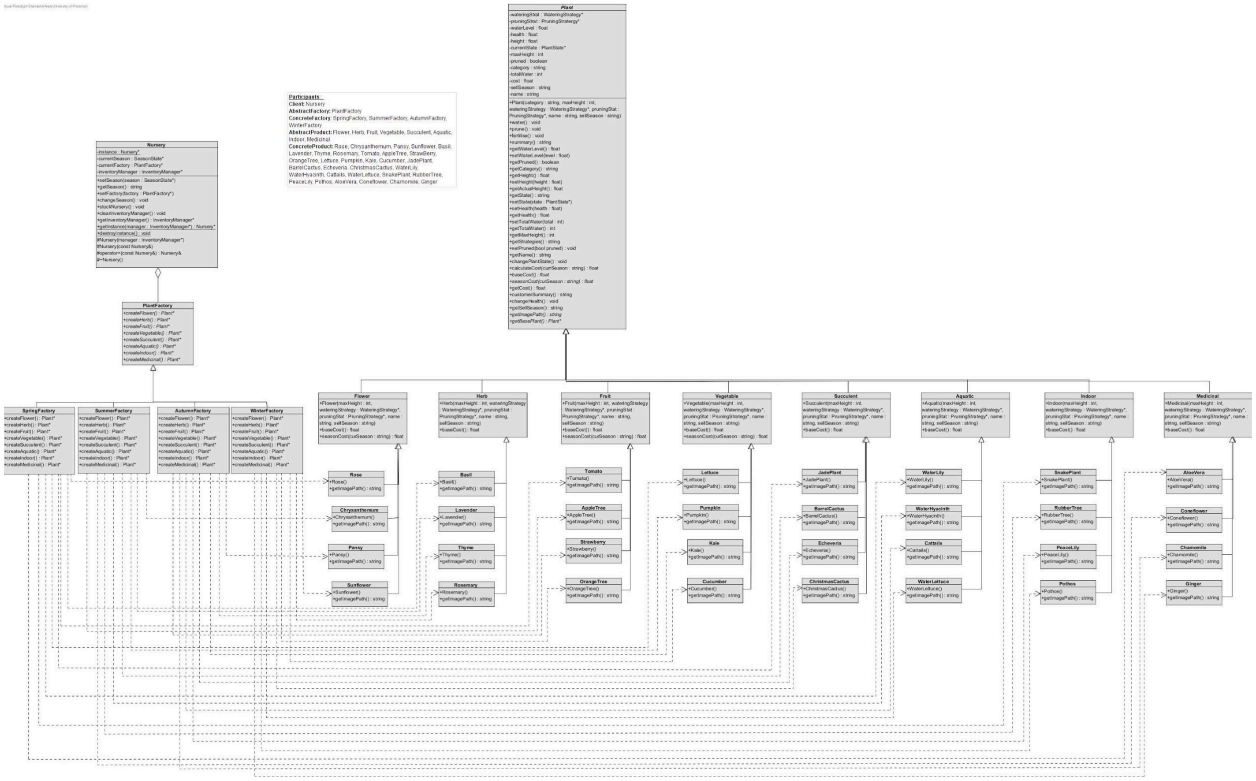
NON-FUNCTIONAL REQUIREMENTS

NFR	Description	Attribute
NFR 1	The system will only allow one instance of a Nursery to exist. This ensures global access to a single, centralized plant inventory, preventing duplication and inconsistencies across the system.	Reliability
NFR 2	The system will allow new plant species to be added without modifying existing code.	Extensibility
NFR 3	The system will only allow authorised staff to modify the nursery inventory.	Security

DESIGN PATTERNS

Categories	Design Pattern
Behavioural	● Template method ● Chain of Responsibility ● Command ● Iterator ● Mediator ● Observer ● State ● Strategy
Creational	● Abstract factory ● Singleton
Structural	● Decorator ● Facade

Abstract Factory



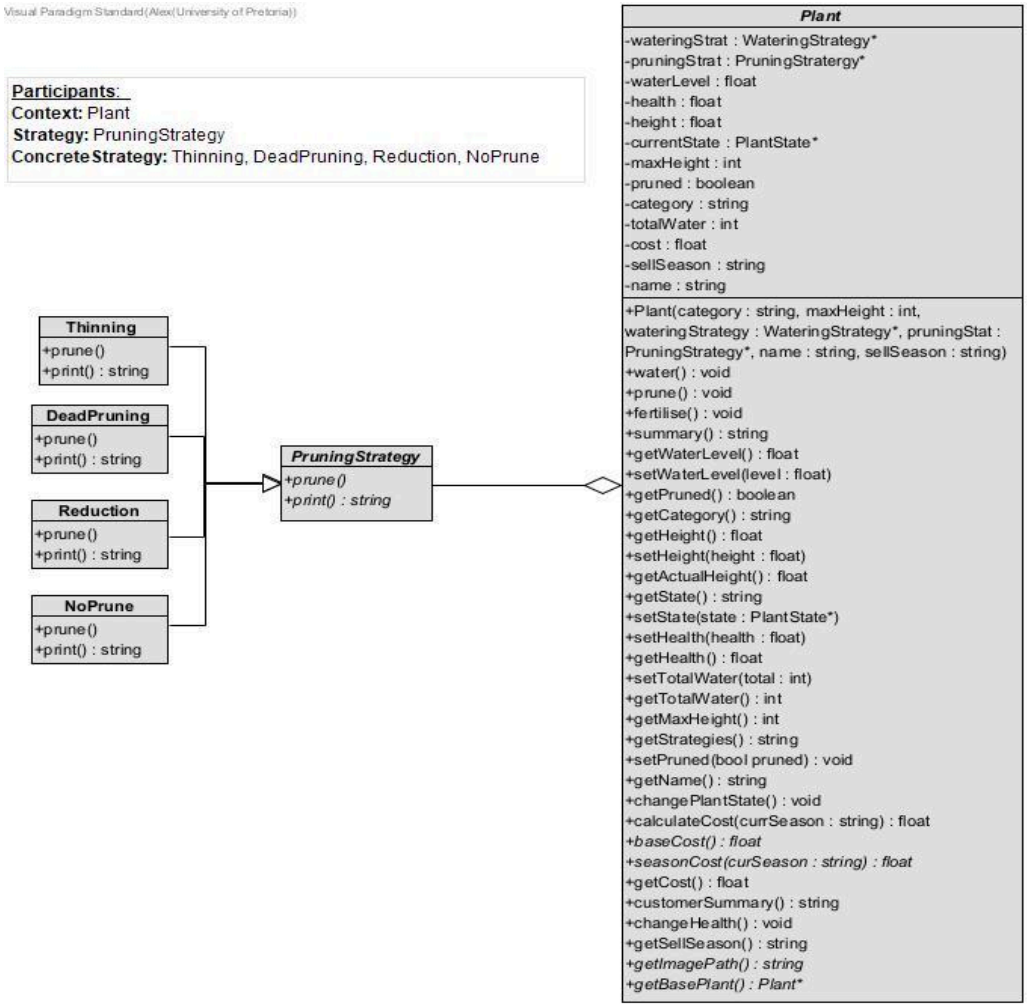
Why & Where: We implemented the Abstract Factory pattern, which is used for the seasonal plant creation system(SpringFactory, SummerFactory, AutumnFactory, WinterFactory) to provide a unified interface (PlantFactory) for creating families of season-related plants (Rose, Lavender, Strawberry, etc.) without specifying the concrete classes.

Functional Requirement: This fulfills FR 1 by allowing for the instantiation of plant families according to season through the use of seasonal factories.

Alternative: Factory Method could be used as a suitable alternative since it also facilitates efficient object creation; however, Abstract Factory is chosen for clear grouping per season.

Strategy

Pruning Strategy

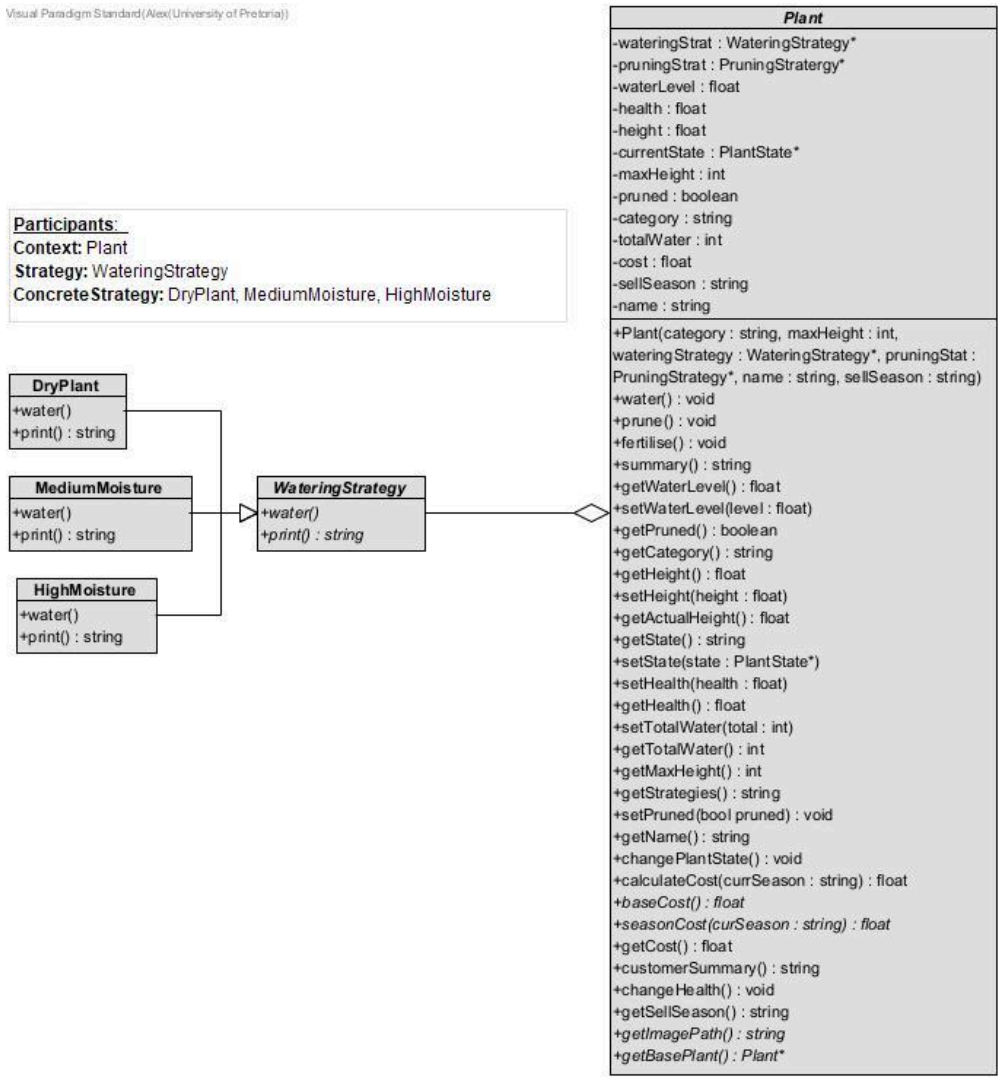


Why & Where: We implemented the Strategy Pattern to encapsulate different pruning algorithms (Thinning, DeadPruning, Reduction, NoPrune) which allows the pruning algorithm to vary between different plants.

Functional Requirement: This fulfills FR 11 by allowing the system to use different strategies for pruning, which allows for easier modification of plant care.

Alternatives: The Template Method pattern could possibly be used here; however, Strategy works best since it uses full algorithm replacement as opposed to parts of the algorithm being interchanged.

Watering Strategy

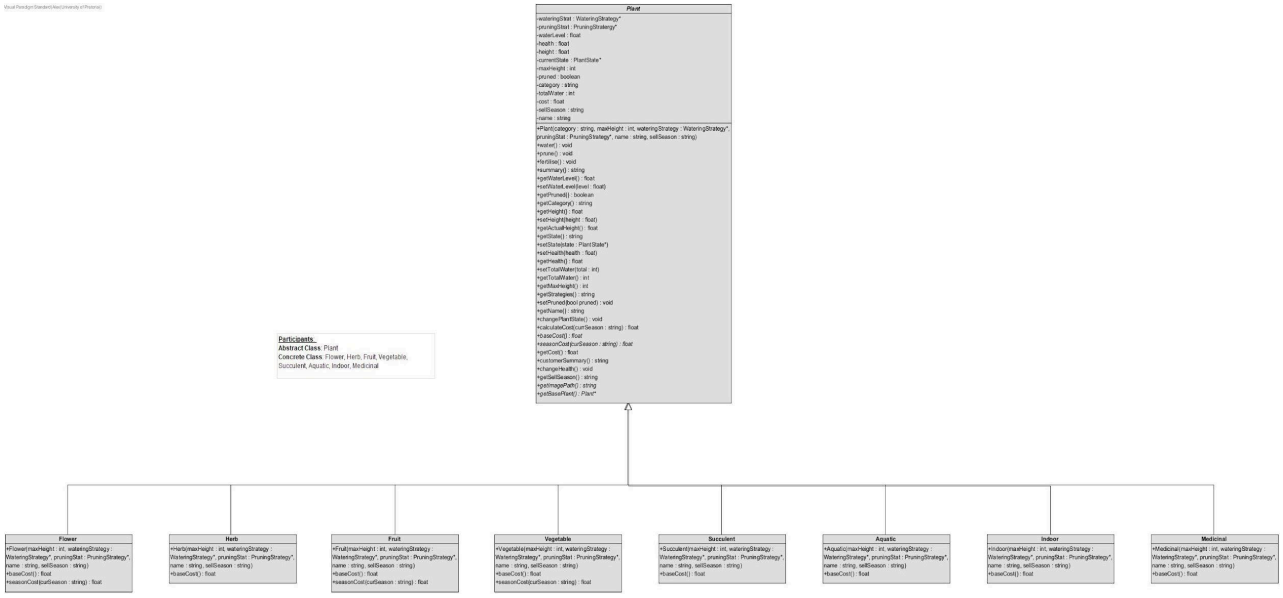


Why & Where: We implemented the Strategy Pattern to encapsulate different watering algorithms (DryPlant, MediumMoisture, HighMoisture), which allows the watering algorithm to vary between different plants.

Functional Requirement: This fulfills FR 12 by allowing the system to use different strategies for watering, which allows for easier modification of plant care.

Alternatives: The Template Method pattern could possibly be used here; however, Strategy works best since it uses full algorithm replacement as opposed to parts of the algorithm being interchanged.

Template Method



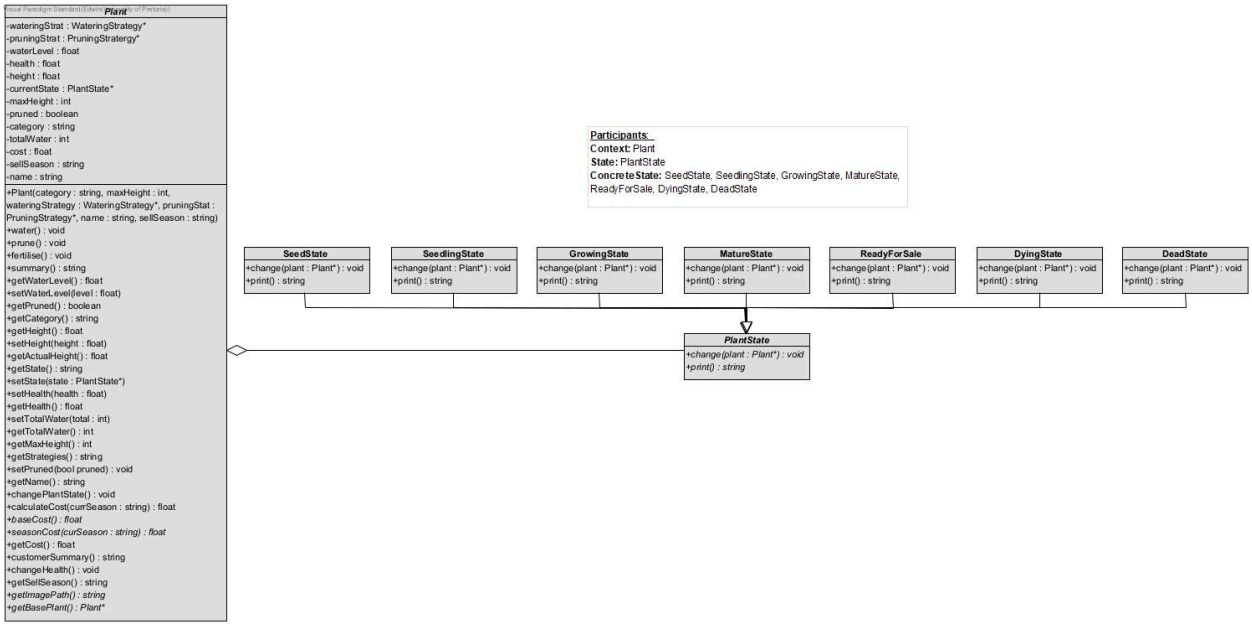
Why & Where: Template Method is used here and defines a fixed skeleton structure used for defining the cost calculation (Plant) while deferring the structure of the cost calculation to each plant type(Flower, Herb, etc.).

Functional Requirement: This fulfills FR 14 by deferring steps in the calculation of the plant cost to the subclasses.

Alternatives: The Strategy pattern could also be used here; however, the step-wise calculation of the cost matches the intent better than Strategy, which relies on the entire algorithm varying.

State

Plant State

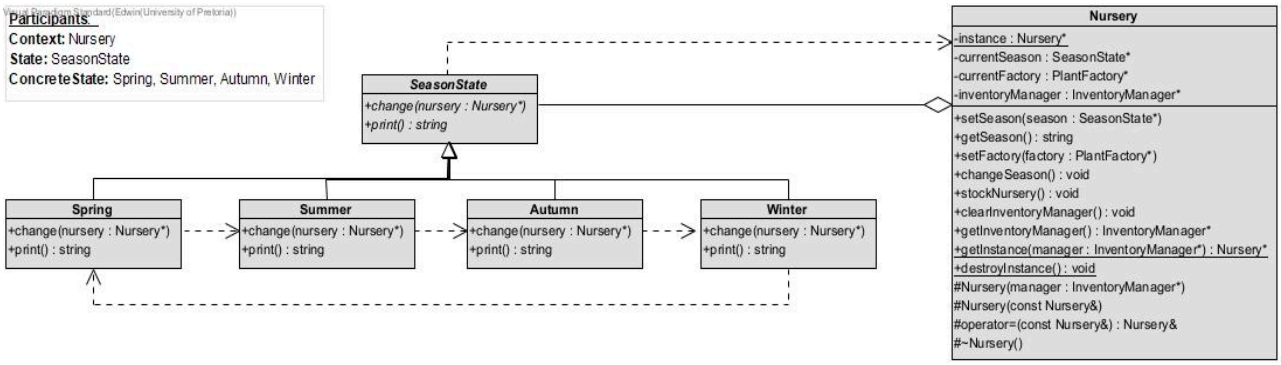


Why & Where: The State pattern is used to allow the plants to alter their internal lifecycle states (SeedState, SeedlingState, GrowingState, etc.), which results in the illusion of a class change.

Functional Requirement: This fulfills FR 9 by allowing for the definition of plant growth states that alter the overall behaviour.

Alternatives: Strategy could be used to interchange algorithms manually; however, State is more suitable since behaviour changes automatically when the internal state changes.

Season State

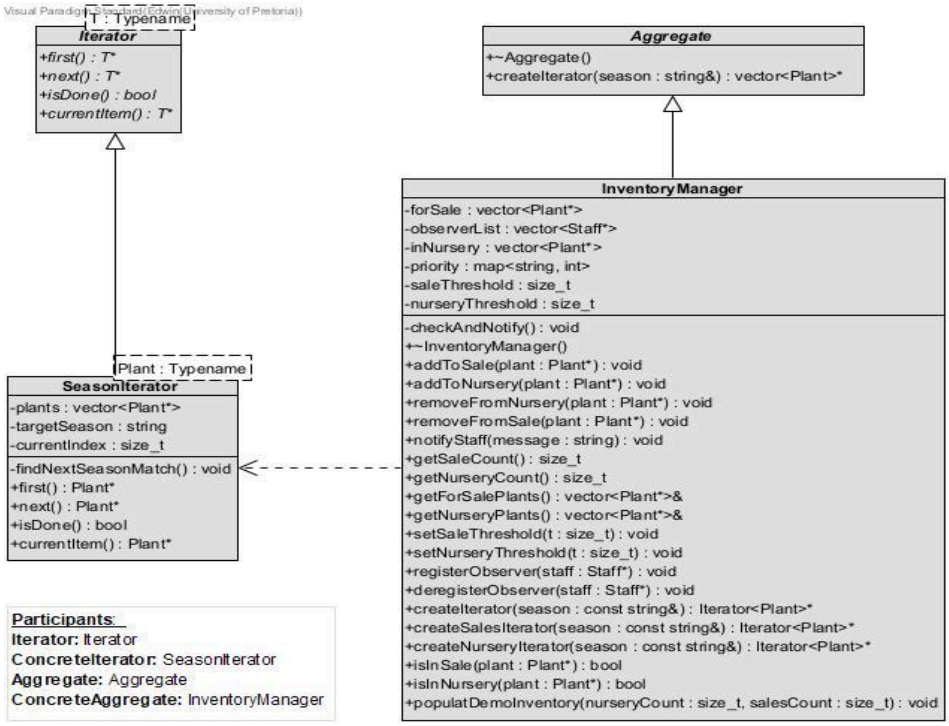


Why & Where: The State pattern is used to allow the nursery to alter its internal season states (Spring, Summer, Autumn and Winter), which results in the illusion of a class change.

Functional Requirement: This fulfills FR 10 by allowing for the definition of nursery season states that allow for different behaviour, ensuring seasonal operations.

Alternatives: Strategy could be used to interchange algorithms manually; however, State is more suitable since behaviour changes automatically when the internal state changes.

Iterator

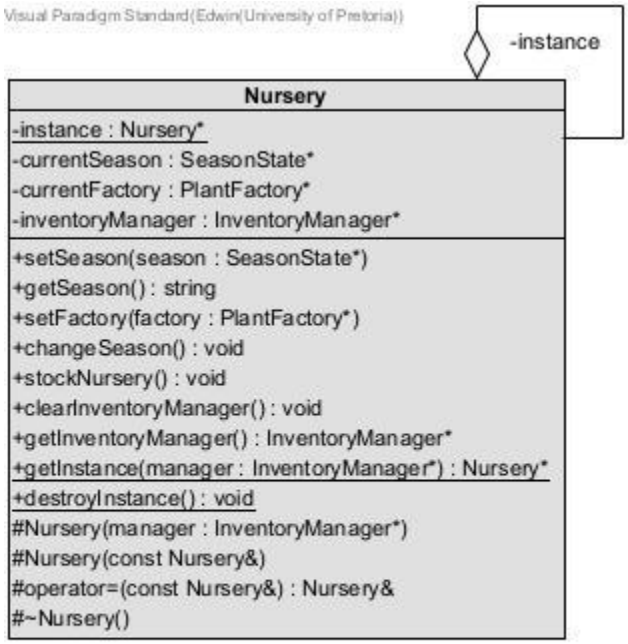


Why & Where: The Iterator Pattern is used in the Concrete Aggregate (InventoryManager) to traverse plants exclusively in the current season without exposing the internal data structure or traversal logic.

Functional Requirement: This fulfills FR 5 by allowing for season-based traversal of plants.

Alternatives: There are no design patterns that could be used to provide a way to access elements of an aggregate sequentially.

Singleton



Participants:
Singleton:Nursery

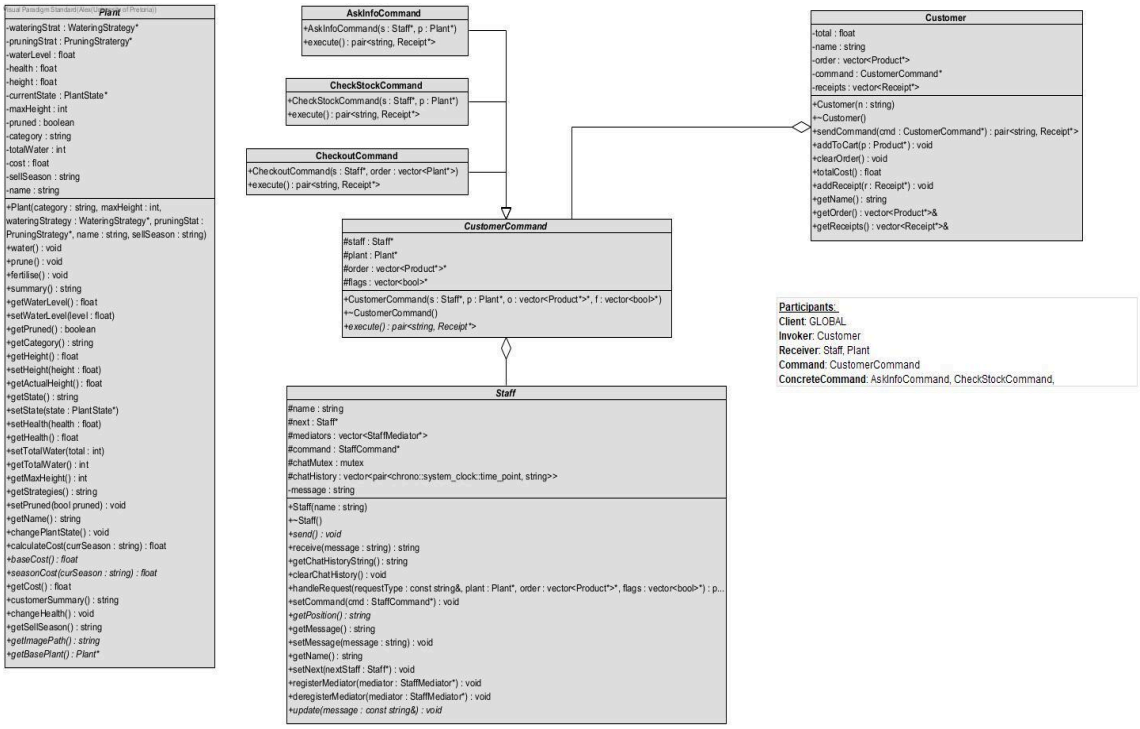
Why & Where: We implemented the Singleton pattern on the Nursery class to enforce a single, globally accessible instance for centralized control over the system's state and inventory.

Functional Requirement: This design fulfills the intent of FR 8 by providing a single shared Nursery instance to prevent data inconsistencies, rather than just the inventory.

Alternatives: You could use the Flyweight pattern to behave like a Singleton. This would happen if your Flyweight Factory was designed to only ever create and return one, single, shared object from its pool, regardless of what "key" you ask for.

Command

Customer Command

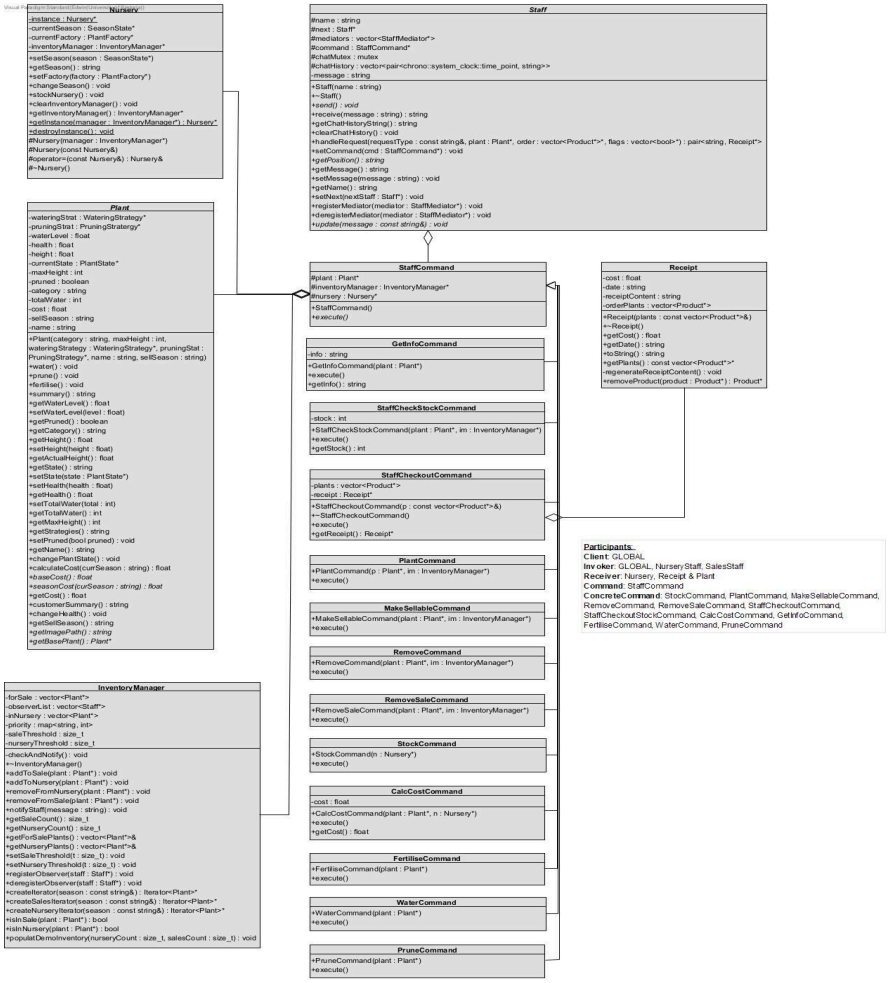


Why & Where: We implemented the Command pattern to encapsulate customer requests like AskInfoCommand, CheckStockCommand, and CheckoutCommand as objects, decoupling the Customer (Invoker) from the Staff (Receiver) who executes the request.

Functional Requirement: This design directly fulfills FR 3, which requires encapsulating actions as command objects to achieve a clear separation between the invoker and the receiver.

Alternatives: The Strategy pattern could also define behaviours, but Command was chosen because it uniquely encapsulates the entire request (the action plus its parameters) as an object, allowing it to be passed and stored.

Staff Command

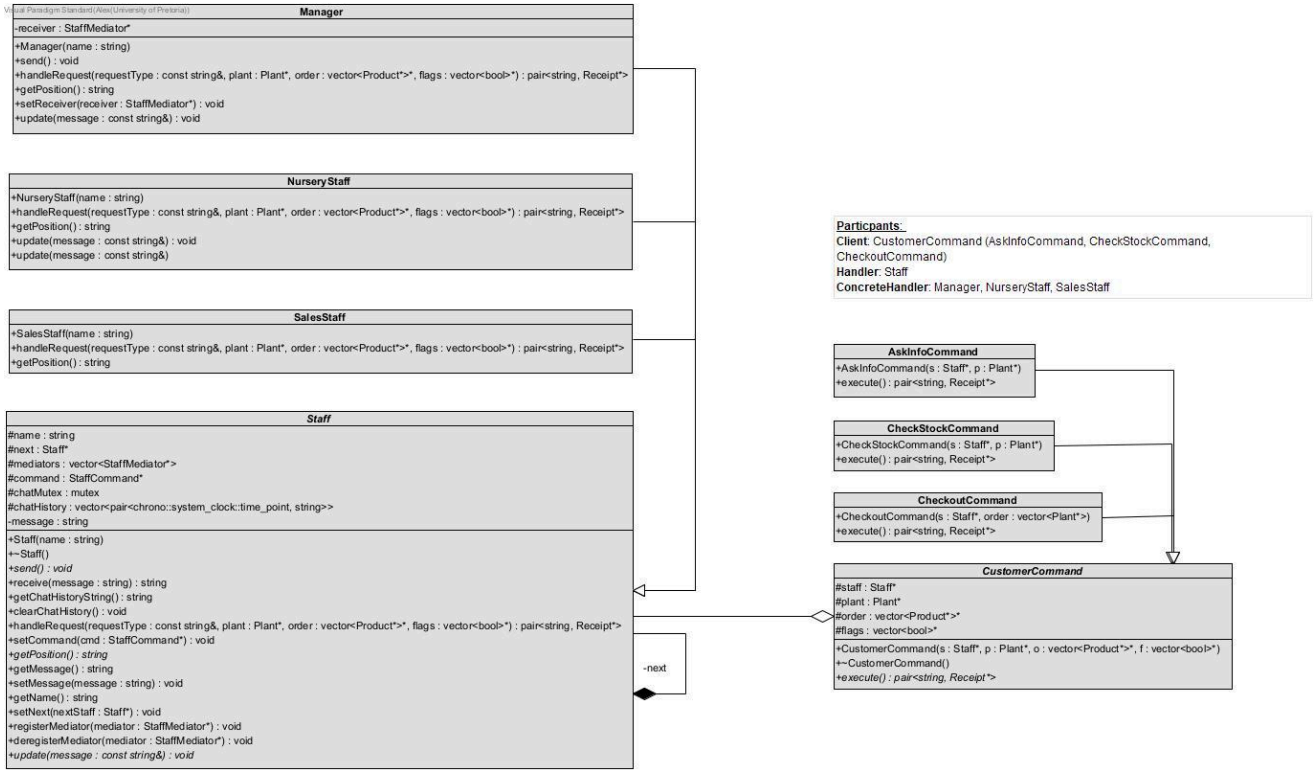


Why & Where: We implemented the Command pattern to encapsulate internal nursery operations like WaterCommand, PlantCommand, and PruneCommand as objects, decoupling the invokers (like Staff) from the receivers (like Plant or InventoryManager).

Functional Requirement: This design directly fulfills FR 3, which requires encapsulating nursery operations like watering and pruning into command objects for a clear separation of concerns.

Alternatives: While Strategy could define how an action is performed, Command was chosen because it encapsulates the entire request to act, allowing it to be triggered by different invokers.

Chain of Responsibility

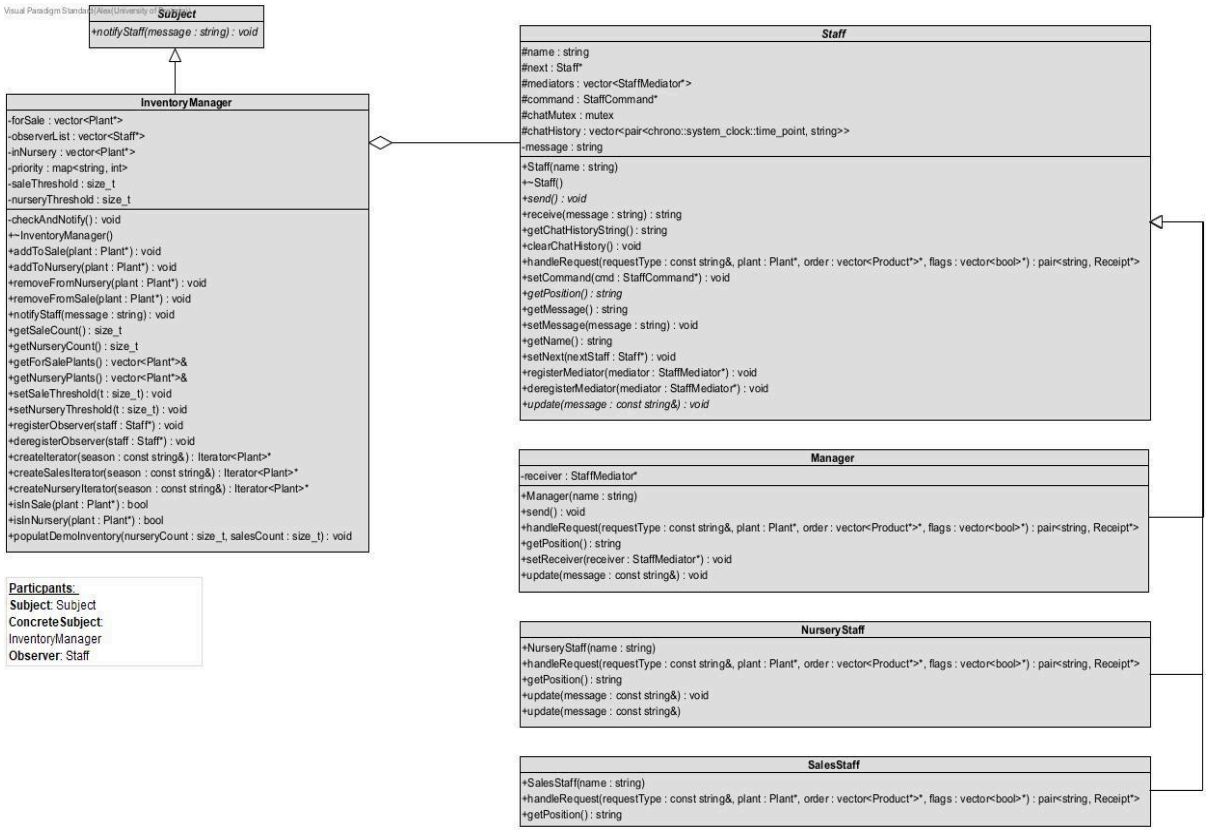


Why & Where: We implemented the Chain of Responsibility pattern using a Staff base class and concrete handlers (Manager, NurseryStaff, SalesStaff) to process customer Command objects, allowing requests to pass along the chain until handled.

Functional Requirement: This directly fulfills FR 2, which requires staff to handle requests based on their level, allowing the request to be passed dynamically among employees.

Alternatives: A Mediator pattern could centralize request handling, but we chose Chain of Responsibility to avoid a single point of failure and allow handlers to be linked in various ways.

Observer

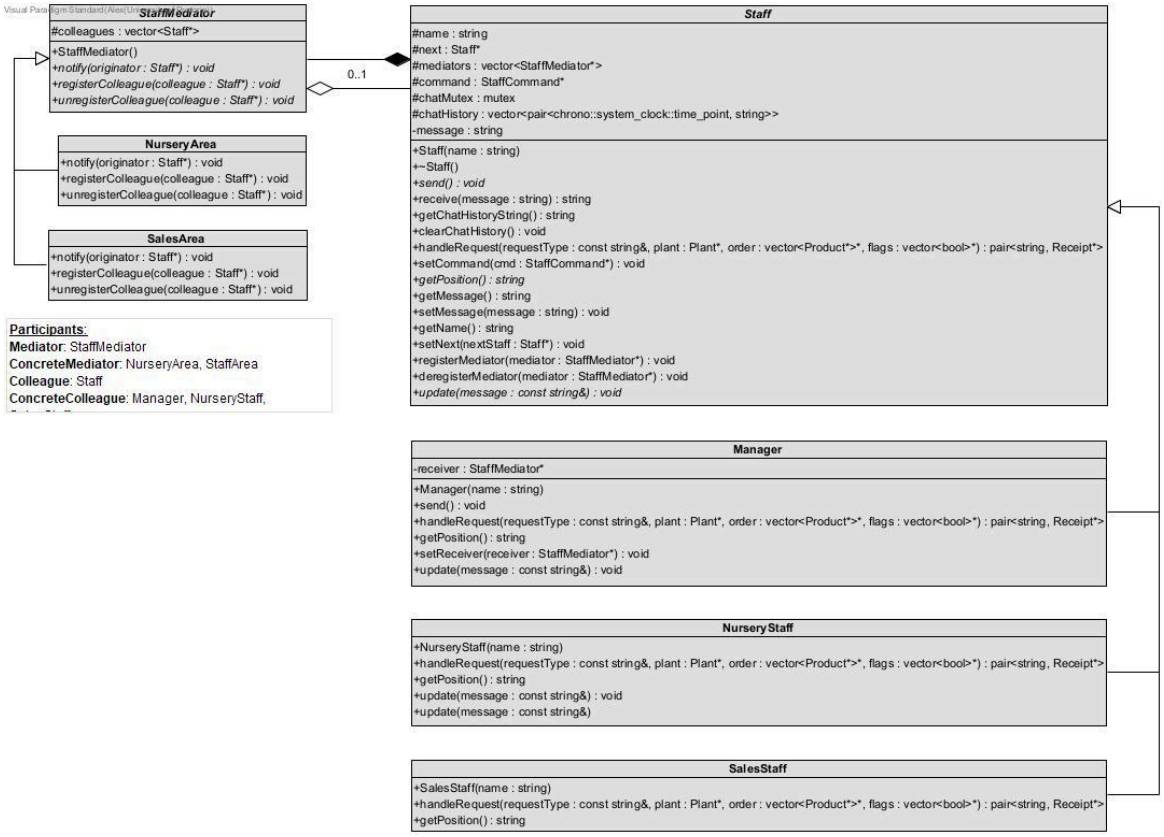


Why & Where: We implemented the Observer pattern with InventoryManager as the Subject and Staff as the Observer, allowing Staff objects to automatically receive updates when inventory levels change.

Functional Requirement: This directly fulfills FR 7, which requires the system to notify relevant staff members when plant inventories fall below a defined threshold.

Alternatives: The Mediator pattern also facilitates communication, but Observer was chosen for its ability to create a one-to-many dependency, allowing any number of Staff observers to subscribe to InventoryManager updates.

Mediator

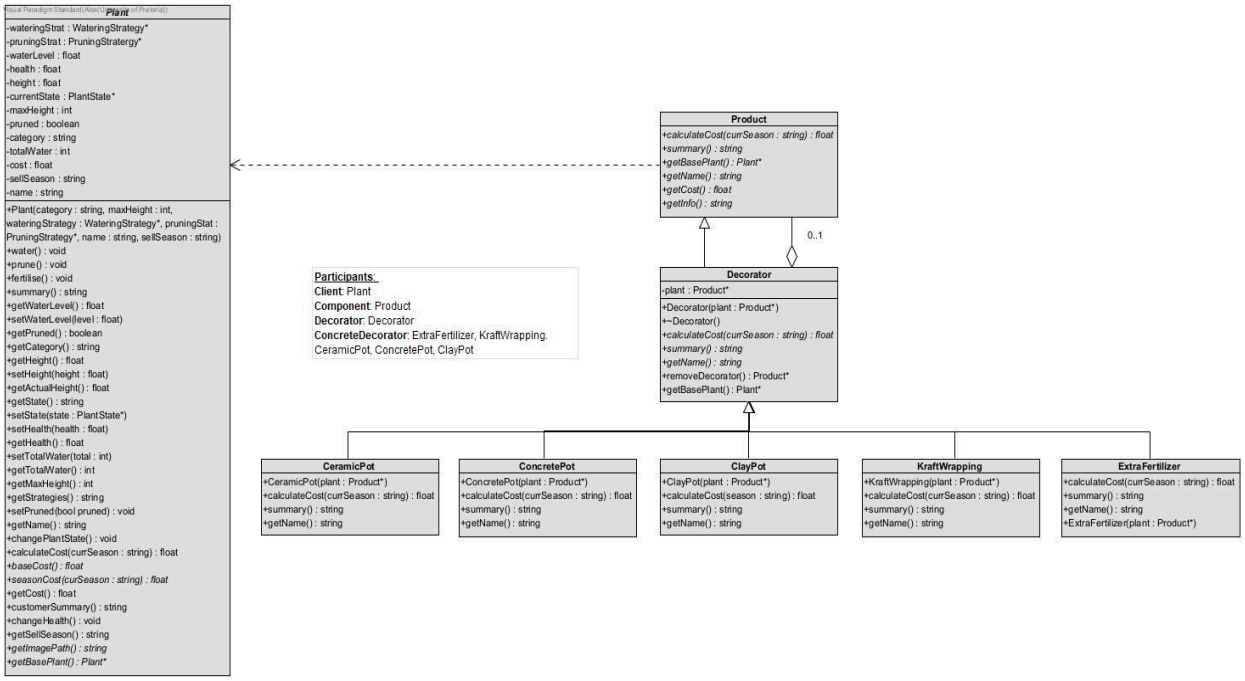


Why & Where: We implemented the Mediator pattern to manage communication between various Staff colleagues (like Manager, NurseryStaff) by having them interact through a central StaffMediator (NurseryArea, SalesArea) instead of coupling them directly to each other.

Functional Requirement: This directly fulfills FR 6, which requires a central mediator to enable communication between different areas and reduce direct coupling between staff.

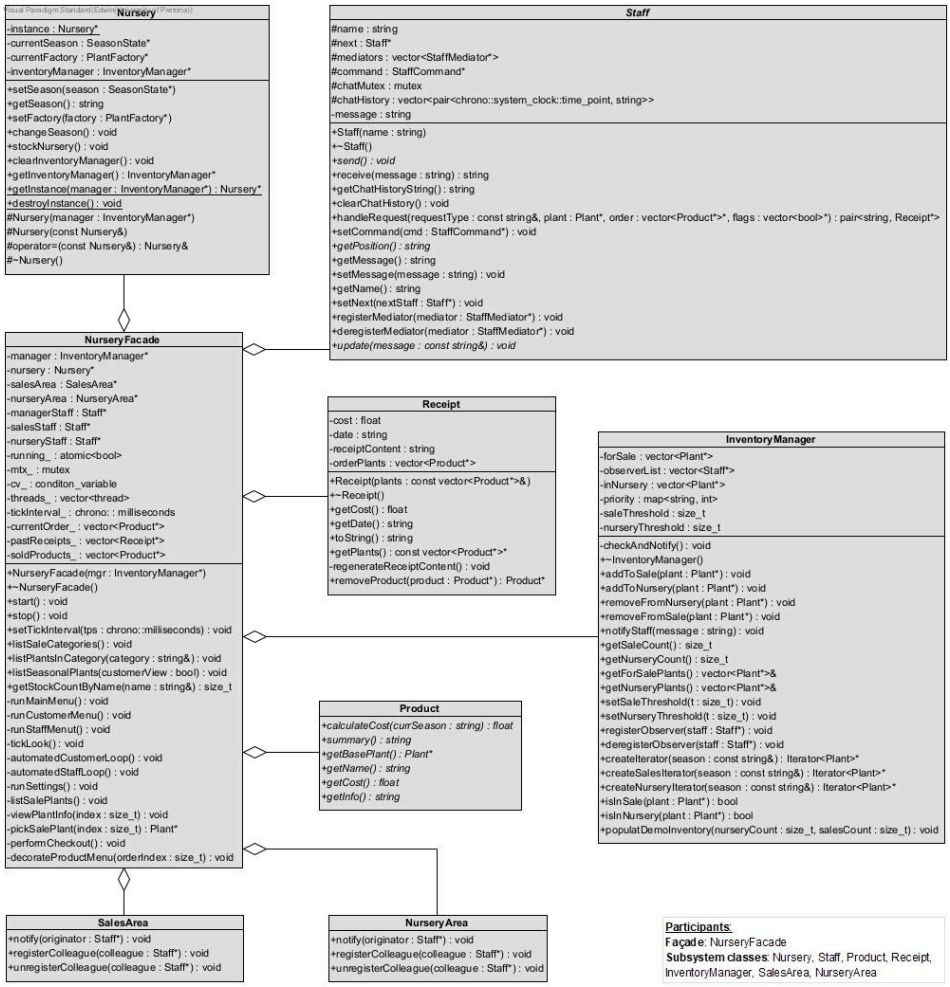
Alternatives: The Observer pattern also manages communication, but Mediator was chosen to handle complex, many-to-many interactions between Staff colleagues, whereas Observer is optimized for one-to-many broadcasts.

Decorator



Why & Where: We implemented the Decorator pattern to dynamically add responsibilities to Plant objects at runtime, using an abstract Decorator class and concrete decorators like KraftWrapping and CeramicPot to wrap the base Product.
Functional Requirement: This design directly fulfills FR 4, which requires the system to allow plants to be decorated based on customer preferences, like gift wrapping.
Alternatives: We chose Decorator over static inheritance (e.g., creating a GiftWrappedPlant subclass) because Decorator allows for greater flexibility, runtime additions, and avoids the "class explosion" that would result from combining all possible decorations.

Façade



Why & Where: We implemented the Facade pattern with the NurseryFacade class to provide a single, simplified interface for clients to perform complex nursery operations, hiding the intricate interactions between subsystem components like Nursery, InventoryManager, and various Areas.

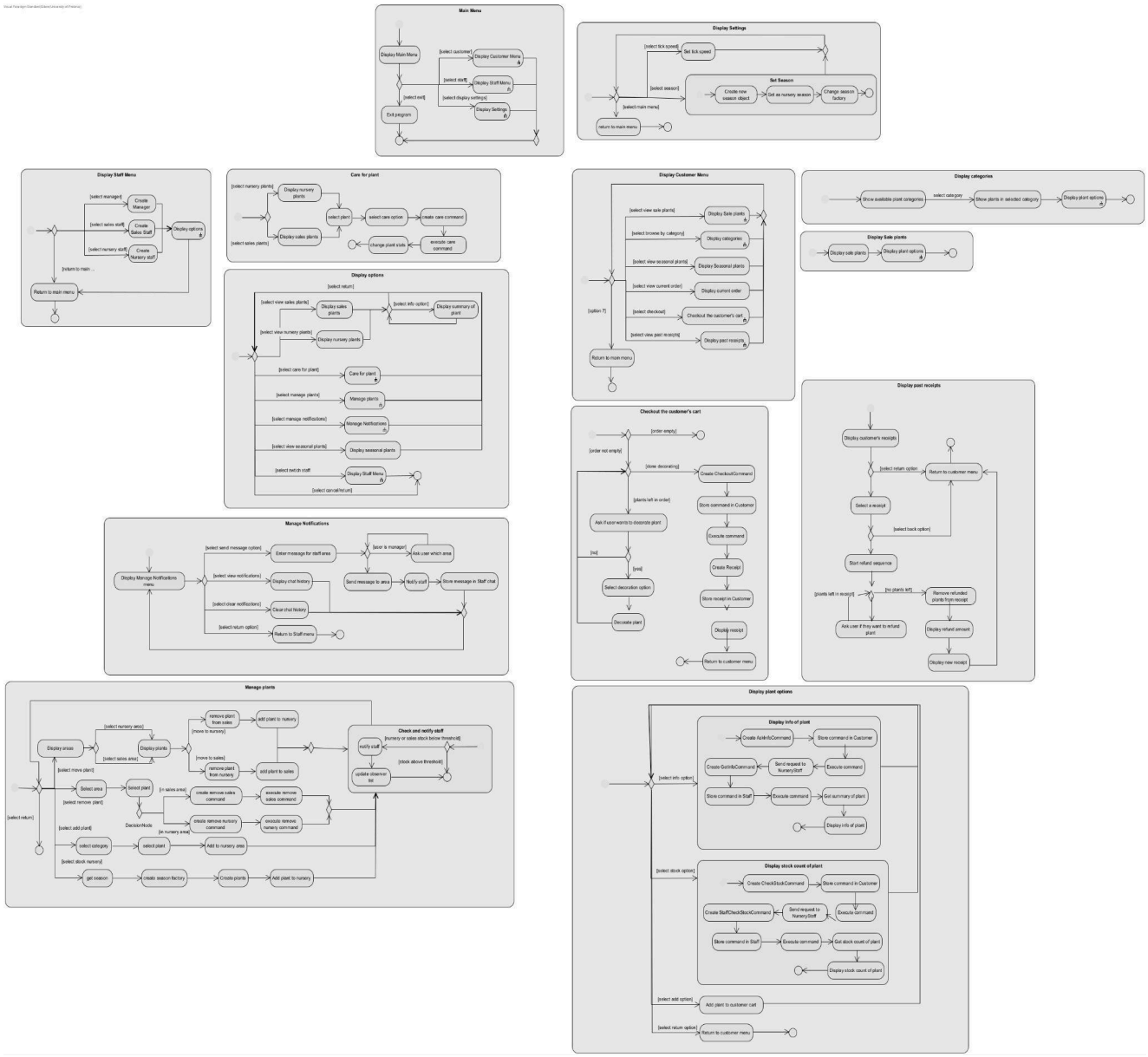
Functional Requirement: This design fulfills FR 13 by creating a unified entry point to the system, simplifying the client's interaction with the nursery's complex subsystems.

Alternatives: The Mediator pattern also centralizes interactions, but we chose Facade to provide a simpler one-way interface to the subsystem, rather than managing the complex many-to-many communications within the subsystem.

UML DIAGRAMS

All the diagrams can be found in the Google Drive folder, in the UML Diagrams subfolder.

High-Level Activity Diagram



GITHUB

Branching Strategy

Types of branches used:

- **main:** The production-ready branch. The 2 commits to this branch are the MVP merge and later the merge of the final project from the dev branch.
- **dev:** This branch acted as the integration point for all completed design pattern implementations. All developments are merged into dev before being merged into main.
- **feature/feature-name:** This branch is used to work on a specific feature of the program. For example, if someone were working on the template method pattern, they would create a feature/template-method branch.
- **fix/fix-name:** This branch is used to indicate someone is issuing a fix to a section of code. For example, if someone were to issue a fix for the abstract factory pattern, they would create a fix/abstract-factory branch.
- **test/test-name:** This branch is used to indicate someone is implementing testing code or unit tests for a section of code. For example, if someone were to implement tests for the decorator pattern, they would create a test/decorator.

Merge Rules

- **Pull requests to main:** This requires the other 4 people in our group to review and approve the pull request, and all unit tests must pass.
- **Pull request to dev:** This requires any 2 other people in the group to review and approve the pull request, and all unit tests must pass.
- **Other branches:** There is no limit on the number of people needed to review or approve the pull request.

Significant changes

- Each design pattern was developed in its own dedicated feature branch. This allowed each group member to work independently without interfering with each other's progress.
- Once a feature branch was completed and tested, it was merged into the dev branch.
- Upon the completion of the MVP(Minimum Viable Product), the dev branch was merged into main, but the dev branch was not deleted to allow for further development of the GUI and TUI.
- When all components (design patterns, GUI, TUI, etc.) were completed, finalised, and tested, the dev branch was merged into main for the final merge.

TESTING

Unit Testing: Google Test vs. doctest

- **Google Test** is a powerful and feature-rich testing framework, offering extensive capabilities like test fixtures, advanced assertions, and robust mocking support.
- **doctest** is a much lighter and simpler framework, known for its fast compilation times and ease of integration (often just a single header file).
- **Our Choice (doctest):** We chose **doctest** because it is significantly simpler to implement and has a much smaller learning curve, which is ideal for our current unit testing needs.

Test Framework Structure

Our testing framework is structured within a single testing folder. Inside this folder, we have individual .cpp files for each design pattern we are testing, such as mediatorTesting.cpp, and each of these files includes the doctest.h header to run the unit tests.

CI/CD Pipeline Implementation

Our CI/CD pipeline is implemented using GitHub Actions for Continuous Integration and GitHub Branch Protection Rules for Continuous Delivery. The entire process is automated to ensure code quality, correctness, and stability before any new code is merged.

1. Continuous Integration (CI)

- Trigger: The CI pipeline is automatically triggered by the main.yml workflow file whenever a pull request is created or updated against the main or dev branches.

- Automated Workflow: The workflow runs on an ubuntu-latest virtual machine and executes the following sequential steps:
    1. Setup: Checks out the repository's code and installs dependencies, including make and valgrind.
    2. Build: Compiles the entire project by running the make all command.
    3. Automated Testing: Runs the make test command, which builds and executes the doctest unit test runner to verify program logic.
    4. Automated Memory Management Check: Runs the make val-all command, which runs valgrind, on both the unit tests and the project build.
- **Gating:** If any of these steps (build, test or memory leak) fail, the pipeline fails, and the pull request is blocked from merging.

2. Continuous Delivery (CD)

- **Manual Gating:** Our Continuous Delivery process is implemented as a manual gate using GitHub's Branch Protection Rules.
- **Process:** A merge is only possible *after* the automated CI pipeline (described above) has passed successfully.
- **Approval Rules:**
    - Merging into the dev branch requires 2 approved reviews.
    - Merging into the main branch requires 4 approved reviews.